FIAP

SOAT — Software Architecture

TECH CHALLENGE — FASE 1

OFICINATRACK

Sistema Integrado de Atendimento e Execução de Serviços

GRUPO 76

Integrante	RM	Discord
Allan de Lima Carvalho Silva	RM 378457	zonahk
Guilherme Azevedo da Trindade	RM 378112	guilhermetrindade97_75307
Laura Henrique Alves	RM 377327	maflexninja
Lucas de Souza Cancela	RM 377972	lucancela
PENDENTE — nome completo de Kayleigh	PENDENTE — RM	kayleighsz_

Linguagem e tecnologias

Java 21; Spring Boot 4.1.1; Spring Web MVC; Spring Security e JWT; Spring Data JPA/Hibernate; PostgreSQL; Flyway; OpenAPI/Swagger; Docker e Docker Compose; JUnit 5; Mockito; Testcontainers; JaCoCo.

São Paulo — 2026

Controle da entrega

Campo	Valor
Curso / fase / grupo	SOAT — Software Architecture / Fase 1 / Grupo 76
Repositório	https://github.com/kayleighszs/oficinatrack
Branch de entrega	master
Commit de referência	02ce391039d5e3616c669e894c05a0fb864416da
Versão do documento	1.0 — 20/10/2026
Vídeo	PENDENTE — inserir URL pública do YouTube

Repositório completo: https://github.com/kayleighszs/oficinatrack

Versão congelada pelo commit:
https://github.com/kayleighszs/oficinatrack/tree/02ce391039d5e3616c669e894c05a0fb864416da

Bloqueios antes da entrega
Preencher nome completo e RM de Kayleigh; inserir link público do vídeo; confirmar Miro sem login; anexar resultado da análise de vulnerabilidades; confirmar branch principal protegida e versão final via PR.

Sumário

- 1. Visão executiva
- 2. Matriz de aderência ao enunciado
- 3. Arquitetura e decisões técnicas
- 4. Domain-Driven Design
- 5. Fluxos e regras de negócio
- 6. APIs e segurança
- 7. Persistência e transações
- 8. Qualidade e evidências
- 9. Execução e demonstração
- 10. Links DDD, roteiro do vídeo e checklist final
- Referências

1. Visão executiva

O OficinaTrack é um MVP de back-end monolítico para substituir anotações e planilhas no atendimento de uma oficina mecânica. A solução concentra cadastros, ordens de serviço, diagnóstico, orçamento, autorização do cliente, consumo de peças, histórico e indicadores em uma API REST documentada.

Problema e resultado esperado

- Reduzir erros de priorização e dar rastreabilidade ao status da OS.
- Evitar perda de histórico de clientes, veículos, orçamentos e serviços.
- Controlar peças e insumos com movimentações auditáveis e baixa atômica.
- Permitir aprovação do orçamento pelo cliente sem acesso administrativo.
- Proteger operações internas com JWT e perfis de acesso.

Escopo entregue

Dimensão	Evidência
Aplicação	88 arquivos Java organizados em nove módulos de negócio/técnicos; 37 endpoints REST.
Banco	PostgreSQL com quatro migrations Flyway e constraints de integridade.
Qualidade	179 testes em 19 classes; build Maven validado; cobertura de linhas de 88,41%.
Operação	Dockerfile, Docker Compose, variáveis de ambiente e Swagger UI.
DDD	Diagramas de domínio, AS IS/TO BE e épicos; quatro quadros Miro; mapeamento código-domínio neste PDF.

2. Matriz de aderência ao enunciado

Status baseado no código e nos artefatos locais em 20/10/2026. Itens pendentes não são apresentados como concluídos.

Requisito	Evidência	Status	Ação/observação
Identificação da entrega	Curso, fase, grupo, integrantes, RMs, Discord e stack na capa.	ATENÇÃO	Nome completo e RM de Kayleigh pendentes.
Repositório público e versionado	https://github.com/kayleighszs/oficinatrack — branch master; commit 02ce391039d5e3616c669e894c05a0fb864416da.	VALIDAR	Confirmar visibilidade pública e proteção da branch principal durante correção.
README e execução local	README, Dockerfile, docker-compose.yml e .env.example.	PARCIAL	README funciona, mas deve receber links do DDD/vídeo e seção arquitetural mais completa.
Vídeo público no YouTube	Demonstração de até 15 minutos vinculada ao commit entregue.	PENDENTE	Inserir URL pública antes de exportar versão final.
Documentação DDD	Quatro quadros Miro e três diagramas Excalidraw no diretório docs/.	VALIDAR	Confirmar acesso anônimo aos quadros; ideal exportar também para PDF/PNG em docs/ddd/.
Funcionalidades obrigatórias	CRUDs, OS, orçamento, consulta pública, estoque, relatório e histórico presentes.	ATENDIDO	Evidência mapeada a pacotes e endpoints neste documento.
JWT e validações	Rotas administrativas autenticadas; roles; CPF/CNPJ e placa validados.	ATENDIDO	JWT HS256, BCrypt, sessão stateless e token público por OS com hash SHA-256.
Testes e cobertura	./mvnw verify: 179 testes; 88,41% de cobertura de linhas no escopo JaCoCo.	ATENDIDO	Execução validada em 20/10/2026 com PostgreSQL via Testcontainers.
Análise de vulnerabilidades	Nenhum relatório versionado foi localizado.	PENDENTE	Executar SCA, revisar achados e incluir relatório/resultado no repositório.
PDF único	Este arquivo reúne identificação, links, decisões e evidências.	ATENDIDO	Substituir pendências antes do envio oficial.

3. Arquitetura e decisões técnicas

3.1 Estilo arquitetural

Monólito modular em camadas. Cada contexto reúne API, aplicação, domínio e infraestrutura. Controllers tratam HTTP; serviços de aplicação demarcam casos de uso e transações; entidades concentram invariantes; repositórios Spring Data isolam persistência. O shared contém validações, tratamento de erros e configuração OpenAPI.

Fluxo de uma requisição
Cliente HTTP → Controller/DTO → Service transacional → Entidade/agregado → Repository → PostgreSQL. Respostas e erros retornam contratos próprios, sem expor entidades JPA diretamente.

3.2 Módulos

Pacote	Responsabilidade	Camadas principais
auth	Login, JWT, usuários e roles	api / application / domain / infra
cliente	Cadastro e validação de documento	api / application / domain / infra / validation
veiculo	Veículo, vínculo com cliente e placa	api / application / domain / infra
catalogoservicos	Serviços, preço e tempo estimado	api / application / domain / infra
estoque	Peças e movimentações	api / application / domain / infrastructure
ordemservico	Core Domain: OS, itens, orçamento e histórico	api / application / domain / infra
publico	Credencial por OS, consulta e decisão	api / application / domain / infra
relatorios	Indicador de tempo médio	api / application
shared	Erros, configuração e validações comuns	config / exception / validation

3.3 Decisão de banco de dados

PostgreSQL foi escolhido por oferecer transações ACID, constraints, locks de linha e consultas relacionais adequadas ao fluxo. OS, itens, versões de orçamento, histórico e movimentações possuem relações fortes; consistência é mais importante que escalabilidade independente no MVP. Flyway mantém evolução reprodutível do schema. A aprovação carrega a OS e bloqueia as peças em ordem determinística, reduzindo risco de corrida e deadlock.

3.4 Tecnologias e justificativas

Tecnologia	Papel
Java 21 / Spring Boot 4.1.1	Base da API e injeção de dependências.
Spring Web MVC	Controllers REST, validação de entrada e serialização JSON.
Spring Data JPA / Hibernate	Mapeamento relacional, repositories e locking.
Spring Security / JJWT	Autenticação stateless, BCrypt e JWT HS256.
PostgreSQL / Flyway	Persistência transacional e migrations versionadas.
springdoc OpenAPI	Swagger UI e contrato navegável.
Docker / Compose	Ambiente reprodutível para API e banco.
JUnit / Mockito / Testcontainers / JaCoCo	Testes isolados, integração real e cobertura mínima.

4. Domain-Driven Design

4.1 Estratégia e Bounded Contexts

O domínio central é a evolução da Ordem de Serviço. Cadastros, estoque, identidade e relatórios dão suporte. No monólito, limites são módulos de código e linguagem, não serviços distribuídos.

Contexto	Tipo	Raiz/centro	Elementos	Responsabilidade
Ordens de Serviço	Core Domain	OrdemServico	Orcamento, itens e histórico	Coordena diagnóstico, orçamento, decisão, execução e entrega.
Clientes	Apoio	Cliente	TipoPessoa; CPF/CNPJ validado	Mantém responsável pelo atendimento e estado ativo.
Veículos	Apoio	Veiculo	Placa	Vincula veículo ao cliente e normaliza placa.
Catálogo de Serviços	Apoio	Servico	valor e tempo estimado	Fonte dos serviços e preços adicionados à OS.
Estoque	Apoio	Peca	MovimentacaoEstoque	Controla saldo, mínimo e trilha de entradas/saídas.
Acesso Público	Apoio	AcessoPublicoOrdemServico	token opaco e validade	Consulta e decisão do cliente sem expor JWT administrativo.
Identidade e Acesso	Genérico	Usuario	Role e JWT	Autentica equipe e aplica autorização por perfil.
Relatórios	Apoio	Consulta	TempoMedioExecucaoResponse	Calcula média entre EM_EXECUCAO e FINALIZADA.

Relações do Context Map

- Clientes → Veículos: cliente fornece identidade usada no vínculo.
- Clientes e Veículos → OS: existência, atividade e pertencimento são validados.
- Catálogo e Estoque → OS: nomes e preços são copiados para itens históricos.
- OS → Estoque: aprovação aciona baixa síncrona e transacional.
- OS → Acesso Público: token limita consulta e decisão a uma OS.
- Identidade protege rotas administrativas; Relatórios lê histórico da OS.

4.2 Modelo tático e relação com o código

Categoria	Implementação	Regra principal
Agregado OS	OrdemServico (raiz); Orcamento; ItemServicoOrdem; ItemPecaOrdem; HistoricoStatusOrdemServico	Mantém transições, versões de orçamento, composição e histórico consistentes.
Agregado Estoque	Peca (raiz); MovimentacaoEstoque	Impede saldo negativo e registra origem manual ou automática.
Entidades	Cliente, Veiculo, Servico, Usuario, AcessoPublicoOrdemServico	Possuem identidade e ciclo de vida persistido.
Objeto de valor	Placa	Normaliza formato antigo/Mercosul e rejeita placa inválida.
Conceitos enumerados	StatusOrdemServico, StatusOrcamento, TipoPessoa, Role, TipoMovimentacao, Origem	Vocabulário fechado usado por regras e persistência.
Repositórios	Interfaces Spring Data por módulo	Abstraem persistência; consultas com lock protegem decisão/estoque concorrente.
Serviços de aplicação	*Service em cada contexto	Orquestram repositórios, transações e chamadas aos agregados.

Nota de fidelidade
Os eventos abaixo descrevem fatos de negócio usados no Event Storming. Na implementação atual, a maioria aparece como mudança de estado/histórico dentro da transação, não como classes publicadas em barramento. A baixa de estoque também é síncrona no serviço de consulta pública.

4.3 Linguagem Ubíqua

Termo	Definição adotada
Ordem de Serviço (OS)	Atendimento de um veículo, do recebimento à entrega.
Diagnóstico	Etapa em que serviços e peças necessários são identificados.
Orçamento	Versão calculada com subtotais de serviços e peças, submetida ao cliente.
Acesso público	Credencial opaca, temporária e vinculada a uma única OS.
Baixa automática	Saída de estoque feita na mesma transação da aprovação do orçamento.
Histórico de status	Registro imutável das transições e respectivos instantes.
Peça/insumo	Material consumível controlado por saldo e estoque mínimo.
Tempo médio de execução	Média entre entrada em EM_EXECUCAO e entrada em FINALIZADA.

4.4 Event Storming consolidado

Fluxo	Atores	Comandos	Eventos/fatos	Políticas
Criação e orçamento da OS	Atendente / mecânico	Cadastrar cliente e veículo; abrir OS; iniciar diagnóstico; incluir serviço/peça; calcular e enviar orçamento	ClienteIdentificado; VeiculoCadastrado; OrdemServicoCriada; DiagnosticoIniciado; OrcamentoCalculado; OrcamentoEnviado	Veículo deve pertencer ao cliente; itens só entram em diagnóstico; orçamento exige ao menos um item; somente versão atual pode ser enviada.
Acompanhamento e decisão	Cliente / atendente / mecânico	Emitir acesso; consultar status/orçamento; aprovar ou recusar; finalizar; entregar	AcessoPublicoEmitido; OrcamentoAprovado ou OrcamentoRecusado; StatusAlterado; OrdemFinalizada; VeiculoEntregue	Token deve estar válido e vinculado à OS; decisão é idempotente; transições seguem máquina de estados.
Gestão de peças e insumos	Gestor / sistema	Cadastrar peça; registrar entrada/saída; aprovar orçamento	EstoqueMovimentado; EstoqueBaixado	Quantidade positiva; saldo não pode ficar negativo; aprovação bloqueia linhas para atualização e registra OS/orçamento na movimentação.

5. Fluxos e regras de negócio

5.1 Fluxo principal da Ordem de Serviço

1. Atendente autentica e recebe JWT.
2. Cliente e veículo são cadastrados; veículo deve pertencer ao cliente.
3. OS é aberta em RECEBIDA; acesso público pode ser emitido e entregue ao cliente.
4. Mecânico inicia diagnóstico; OS passa a EM_DIAGNOSTICO.
5. Serviços e peças são incluídos com quantidade positiva e snapshot de preço.
6. Sistema calcula versão do orçamento e soma subtotais com escala monetária de duas casas.
7. Atendente envia apenas o orçamento atual; OS passa a AGUARDANDO_APROVACAO.
8. Cliente consulta e aprova/recusa usando token opaco vinculado à OS.
9. Na aprovação, peças são bloqueadas e debitadas; movimentações são registradas; OS passa a EM_EXECUCAO.

10 Mecânico finaliza; atendente confirma entrega; histórico registra todas as mudanças.

5.2 Máquina de estados

Origem	Comando	Destino	Proteção
—	Abrir OS	RECEBIDA	Cliente/veículo válidos
RECEBIDA	Iniciar diagnóstico	EM_DIAGNOSTICO	Gestor ou mecânico
EM_DIAGNOSTICO	Enviar orçamento	AGUARDANDO_APROVACAO	Orçamento atual calculado
AGUARDANDO_APROVACAO	Aprovar	EM_EXECUCAO	Token válido + estoque
AGUARDANDO_APROVACAO	Recusar	ORCAMENTO_RECUSADO	Motivo obrigatório
EM_EXECUCAO	Finalizar	FINALIZADA	Gestor ou mecânico
FINALIZADA	Entregar	ENTREGUE	Gestor ou atendente

5.3 Invariantes observadas no código

- Não duplicar o mesmo serviço ou peça na OS.
- Não calcular orçamento vazio nem enviar versão antiga.
- Itens só entram durante diagnóstico; transições incompatíveis retornam conflito.
- Recusa exige motivo; aprovação repetida é idempotente e decisão oposta gera conflito.
- Peça inativa ou saldo insuficiente impede aprovação; transação desfaz decisão e movimentações.
- Token público expira, pode ser revogado, é exibido somente na emissão e armazenado como SHA-256.
- @Version na OS e locks pessimistas ajudam a tratar concorrência.

6. APIs e segurança

6.1 Catálogo de endpoints

Área	Rotas	Acesso	Finalidade
Autenticação	POST /auth/login	Público	Valida credenciais e emite JWT.
Clientes	POST /clientes; GET /clientes; GET /clientes/{id}; PUT /clientes/{id}; PATCH /clientes/{id}/inativar	JWT	Cadastro, busca, atualização e inativação.
Veículos	POST /veiculos; GET /veiculos; GET /veiculos/{id}; PATCH /veiculos/{id}	JWT	Cadastro vinculado ao cliente, consulta e atualização/inativação.
Serviços	POST /servicos; GET /servicos; GET /servicos/{id}; PUT /servicos/{id}; PATCH /servicos/{id}/inativar	JWT	Catálogo de serviços.
Peças	POST /pecas; GET /pecas; GET /pecas/{id}; PUT /pecas/{id}; PATCH /pecas/{id}/estoque	JWT	Cadastro, filtros e movimentação manual.
OS — fluxo	POST /ordens-servico; POST /{id}/diagnosticos; POST /{id}/itens-servico; POST /{id}/itens-peca; POST /{id}/orcamentos; POST /{id}/orcamentos/{orcamentoId}/envios; POST /{id}/finalizacoes; POST /{id}/entregas	JWT	Fluxo administrativo e transições.
OS — consultas	GET /ordens-servico; GET /ordens-servico/{id}; GET /ordens-servico/{id}/historico-status	JWT	Lista paginada com filtros, detalhe e histórico.
Acesso da OS	POST /ordens-servico/{ordemId}/acessos-publicos; DELETE /ordens-servico/{ordemId}/acessos-publicos/{acessoId}	JWT	Emite token uma vez e permite revogação.
Cliente	GET /publico/ordens-servico/{numeroOs}/status; GET /publico/ordens-servico/{numeroOs}/orcamentos/{orcamentoId}; POST /publico/ordens-servico/{numeroOs}/orcamentos/{orcamentoId}/decisoes	Token da OS	Consulta e decisão do orçamento.
Relatórios	GET /relatorios/tempo-medio-execucao	JWT/GESTOR	Média do tempo de execução.

6.2 Autenticação e autorização

- Sessão stateless; CSRF desabilitado; filtro JWT antes do filtro de usuário/senha.
- Somente login, /publico/** e OpenAPI são liberados sem JWT administrativo.
- GESTOR, ATENDENTE e MECANICO aplicados com @PreAuthorize.
- Senhas com BCrypt; segredo e expiração JWT vêm do ambiente.
- Token público usa 32 bytes aleatórios, Base64 URL-safe, validade e revogação.
- Erros centralizados e DTOs evitam contratos inconsistentes.

6.3 Validações sensíveis

Dado	Tratamento
CPF/CNPJ	Remove pontuação, rejeita sequências e valida dígitos verificadores.
Placa	Maiúsculas, sem espaços/hífen, padrão antigo ou Mercosul.
Valores/quantidades	BigDecimal; quantidades positivas; saldo insuficiente bloqueia.
Vínculos	Veículo ativo precisa pertencer ao cliente ativo da OS.

7. Persistência e transações

O schema é criado por quatro migrations. Tabelas principais: usuarios, clientes, veiculos, servicos, pecas, movimentacoes_estoque, ordens_servico, itens_servico_ordem, itens_peca_ordem, orcamentos, historico_status_os e acessos_publicos_os.

Decisões de consistência

- Foreign keys e índices sustentam integridade e consultas.
- Versão do orçamento é única por OS; número da OS e documento/placa normalizados são únicos.
- Decisão carrega OS para atualização; peças são bloqueadas por IDs ordenados.
- Aprovação, baixa, movimentações e EM_EXECUCAO compartilham transação.
- Histórico é criado em cada transição; relatório deriva dos instantes persistidos.

Migrations

Versão	Arquivo	Conteúdo
V1	V1__baseline.sql	Cadastros, estoque e constraints iniciais.
V2	V2__perfis_acesso.sql	Papéis de acesso e usuários.
V3	V3__ordens_servico.sql	OS, itens, orçamento e histórico.
V4	V4__acesso_publico_e_baixa_atomica.sql	Tokens, índices e rastreio de baixa automática.

8. Qualidade e evidências

8.1 Resultado reproduzido

Build aprovado
Comando: ./mvnw -q verify. Exit code 0; 179 testes; 0 falhas/erros; JaCoCo 717/811 linhas = 88,41%.
Integração executada contra PostgreSQL via Testcontainers.

8.2 Estratégia de testes

Nível	Exemplos	O que protege
Domínio	OrdemServicoTest, PecaTest, MovimentacaoEstoqueTest	Transições, orçamento, saldo e movimentações.
Aplicação	ClienteServiceTest, VeiculoServiceTest, OrdemServicoServiceTest, ServicoServiceTest	Orquestração, validações, filtros e erros.
API	ClienteControllerTest, ServicoControllerTest, AuthControllerTest	Contratos HTTP, validação e segurança.
Integração	ConsultaPublicaIntegracaoTest	Aprovação, baixa atômica, entrega e relatório.
Gate	JaCoCo no phase verify	Build falha abaixo de 80% no escopo.

8.3 Segurança de dependências

Pendência de entrega
O enunciado exige relatório ou resultado de análise de vulnerabilidades. Não foi encontrado artefato versionado. Executar SCA, revisar achados, guardar relatório em docs/security/ e referenciá-lo no README e neste PDF.

9. Execução e demonstração

9.1 Pré-requisitos

- Docker e Docker Compose para execução completa.
- Alternativa: Java 21 e Docker para PostgreSQL.
- .env criado de .env.example, sem versionar segredos.

9.2 Execução

```
cp .env.example .env
docker compose up --build
# API: http://localhost:8080
# Swagger: http://localhost:8080/swagger-ui.html
# Testes: ./mvnw verify
```

9.3 Credenciais de demonstração

O DataSeeder cria usuários de demonstração a partir de variáveis de ambiente. Não publicar senhas reais; no vídeo, mostrar configuração e autenticação com valores descartáveis.

10. Links DDD, vídeo e checklist

10.1 Quadros colaborativos

Event Storming: https://miro.com/welcomeonboard/cEpNN1pQTGxgSzJieDEwLzN3ZjRmNEZWM2xiOW00S0tZOVBJaUtwWVozZFZIbHQwY3ZhWGNGZE1sU0tVakJYTzV4U2w4L0F6T0x0bFdkdnFhVXMvUU9ac3lwc1RHVEJITUtyOTRFWUp1bG9xT0RZeUs3bUxERS9jWHlzdzBCTWtNakdSWkpBejJWRjJhRnhhb1UwcS9BPT0hdjE=?share_link_id=89439599795

Bounded Contexts: https://miro.com/welcomeonboard/Y0pVSFJzV2NaUHhGSjExaks2S2FEelptVHFBQ0lXMFQrbmVBSGF2SWVuWkVXditBV2o4U0F3dHVyT0hndVlRWXdHRHVHMFdMOG12cXpoZEZDeHVmMDZmblptTUx0VHBtRml6bGd6bjNnbE8xS2EyaWFaY3UrZnhUNzJFRjk4M0JhWWluRVAxeXRuUUgwWDl3Mk1qRGVRPT0hdjE=?share_link_id=835649372749

Domain Storytelling: https://miro.com/welcomeonboard/dU1scWJkWVFXUitoU1lWVlNua3B5bk9qWldGOGJyelg4ZUdYMWNyR2lLb1NXTGEybHlYTmpFZVhkaFdxcWFtcGN4cmN2VjFLNVFoenZQblJGVEQyTzZmblptTUx0VHBtRml6bGd6bjNnbE1QT1QyOUQ1V05ISE5LTWJoaThjTWZ0R2lncW1vRmFBVnlLcVJzTmdFdlNRPT0hdjE=?share_link_id=454382467708

Tactical Design: https://miro.com/welcomeonboard/SWpmRTdqWk1takdKaFNEaFB6dEZONTM5VHMrdUVwMSs3cGJpcHNYaFVHVitURW5rbmRuWWxueTR3UzJPaTYxRUl4ZVlKWDdEVSs3dFB6WS92N3FkSGFmblptTUx0VHBtRml6bGd6bjNnbE96WkQ0Mk1maDJTNURNSmFPd2svTkRBd044SHFHaVlWYWk0d3NxeHNmeG9BPT0hdjE=?share_link_id=2671986207466

Artefatos locais

- docs/ddd-oficina-mecanica.excalidraw — atores, subdomínios, context map e eventos.
- docs/fluxo-negocio-as-is-to-be.excalidraw — AS IS e TO BE.
- docs/fluxo-tecnico-historias.excalidraw — épicos, histórias e dependências.
- docs/Fluxo-api-mapeamento.md — guia técnico; revisar trechos antigos antes da entrega.

Acesso para correção
Abrir cada Miro em janela anônima. Exportar quadros para PDF/PNG e versionar em docs/ddd/, conforme enunciado.

10.2 Vídeo

PENDENTE — URL do YouTube
Inserir link completo e público, até 15 minutos, correspondente ao commit 02ce391039d5e3616c669e894c05a0fb864416da

Roteiro sugerido (até 15 min)

Tempo	Conteúdo
0:00–1:00	Equipe, problema, proposta e commit.
1:00–3:00	Arquitetura, camadas e PostgreSQL.
3:00–5:00	DDD: Event Storming, contextos e agregados.
5:00–11:00	Swagger: fluxo completo da OS e estoque.
11:00–12:30	Consulta pública, histórico e relatório.
12:30–14:00	Testes, cobertura e Testcontainers.
14:00–15:00	Docker, vulnerabilidades e encerramento.

10.3 Checklist final

- [ ] Completar nome e RM de Kayleigh.
- [ ] Inserir vídeo público e testá-lo anonimamente.
- [ ] Confirmar repositório público, branch protegida e commit final via PR.
- [ ] Exportar Miro para docs/ddd/ e testar links.
- [ ] Gerar e revisar relatório de vulnerabilidades.
- [ ] Atualizar README com arquitetura, banco, DDD e vídeo.
- [ ] Corrigir documentação interna incompatível com código.
- [ ] Rodar ./mvnw verify no commit final.
- [ ] Confirmar ausência de segredos.
- [ ] Abrir hyperlinks do PDF final.

Referências

Enunciado oficial da fase: docs/Fase 1 - Tech Challenge.pdf

Domain-Driven Design: https://martinfowler.com/bliki/DomainDrivenDesign.html

Bounded Context: https://martinfowler.com/bliki/BoundedContext.html

Spring Security — JWT: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html

PostgreSQL — Transactions: https://www.postgresql.org/docs/current/tutorial-transactions.html

OWASP Dependency-Check: https://owasp.org/projects/dependency-check

Fontes primárias do projeto

- Código-fonte e testes no commit informado.
- README.md, pom.xml, application.yaml, Dockerfile e docker-compose.yml.
- Migrations V1–V4 e artefatos DDD em docs/.
- Relatório JaCoCo gerado em 20/10/2026.